AI Agent Ghostwork Detector: Forecasting Early Task Failures and False Completion Claims from SWE-bench Trajectories

Team G07
Dun-Jie Yang, Charlie Wang, Mo-Ty Wang, Wei-Chia Huang
Department of Computer Science, National Yang Ming Chiao Tung University
https://github.com/calculusfkyou/nycu-data-mining-assignment-2-115-1

Abstract

Autonomous software engineering agents frequently terminate declaring task completion, yet their generated patches silently fail external regression suites—a failure mode we formalize as *ghostwork*. In continuous integration (CI/CD) pipelines, deploying unverified agent commits risks catastrophic production regressions. We present an end-to-end anomaly detection framework to forecast agent task failures directly from execution trajectories before code integration. Evaluated on 284 real trajectories from SWE-bench Lite across five random seeds, our Tier 2 bottleneck Autoencoder—trained strictly on successful workflows without failure labels—achieves a PR-AUC of 0.976 ± 0.003 and cost-sensitive F_2 of 0.952 ± 0.006. This performs comparably to a fully supervised XGBoost upper bound (0.977 ± 0.000, paired t-test $p = 0.565$) while significantly outperforming a rule-based heuristic (0.964 ± 0.000, $p < 0.001$). Ablations prove that trajectory interaction dynamics capture critical failure signals lost when inspecting final patch diffs alone (ΔPR-AUC $= -0.012$), cutting silent bug escapes by 70%.

1 Introduction

As Large Language Model (LLM) agents are integrated into production software engineering workflows, detecting silent execution failures is paramount. When autonomous agents operate in repository environments, they execute multi-step tool sequences: opening files, editing lines, executing test scripts, and inspecting compiler diagnostics (Yang et al., 2024). However, agent self-evaluations are notoriously unreliable; agents frequently conclude their reasoning chains declaring full success, while the underlying bug remains unresolved.

Scenario and Stakeholders. Our system targets automated CI/CD gatekeepers and agent platform operations teams. When an agent concludes a task, the detector intercepts the trajectory before automated pull request creation. If the anomaly score exceeds decision threshold τ, the pipeline triggers an automated **quarantine alert**, preventing automated commits to release branches and dispatching the execution trace to human triage.

Motivation. Detecting agent task failure is technically challenging due to high variance in human codebases, diverse tool interfaces, and severe operational costs of false negatives. Prior monitoring platforms (e.g., LangSmith, Braintrust) rely on hand-crafted heuristic rules or expensive LLM judge evaluations that cost upwards of $0.05 per trace (Galea and Collaborators, 2026). Developing a zero-leakage, low-latency (< 0.01 ms) semi-supervised detector that models the manifold of healthy problem-solving provides a scalable operational safeguard.

Definition of Anomaly. We define the unit of detection as a complete task execution trajectory $x \in \mathbb{R}^D$.

- **Normal Sample:** The agent explores the codebase, isolates the bug, applies a focused diff, and verifies that regression tests pass (*e.g.*, `astropy-14182`, resolved in 18 steps with 0 lingering errors).
- **Anomaly Sample (Ghostwork):** The agent enters recursive tool loops, encounters cascading tracebacks, claims completion (*"the issue has been fixed and verified, submitting"*), but fails external unit tests (*e.g.*, `astropy-14365`, 36 steps, 8 error cascades, unverified submit).

Contributions.

1. We formulate the task of *Agent Ghostwork Detection* on 284 real SWE-bench Lite trajectories under strict partition hygiene.

Work	Data Domain	Method Family	Paradigm	PR-AUC
Galea and Collaborators (2026)	ATFD Multi-Task	Heuristic Rules	Unsup.	0.742
Galea and Collaborators (2026)	ATFD Multi-Task	Claude Sonnet 4	LLM Judge	0.914
Chen and Guestrin (2016)	SWE-bench Lite	Supervised Trees	Supervised	0.977
Ours (T2)	**SWE-bench Lite**	**Deep Autoencoder**	**Semi-Sup.**	**0.976**

Table 1: Positioning comparison against literature monitoring methods on agent trajectories.

2. We prove empirically that full trajectory interaction dynamics dominate static final-patch diffs, improving PR-AUC by $+0.012$ and F_2 by $+0.019$.

3. We demonstrate that a semi-supervised bottleneck Autoencoder trained solely on normal executions achieves 0.976 PR-AUC, matching supervised baselines without requiring labeled failures.

2 Related Work

Data Category and Literature. Our study operates within **Category 4: Text / Logs**, specifically structured execution logs and interaction traces. Autonomous coding agents represent a rapidly expanding paradigm. Jimenez et al. (2024) established the SWE-bench benchmark for real-world GitHub issue resolution. Yang et al. (2024) introduced SWE-agent, designing agent-computer interfaces (ACI) for code search and execution. Recently, Galea and Collaborators (2026) introduced the ATFD benchmark, evaluating monitoring tools across multi-turn trajectories and noting that heuristic baselines suffer high false alarm rates while LLM judges incur prohibitive monetary latency. In anomaly detection, deep one-class classification (Ruff et al., 2018) and tree isolation (Liu et al., 2008) serve as established reconstruction paradigms.

Positioning & 3-Tier Selection. We benchmark three representative tiers: Tier 1 (Naive Heuristic) implements ATFD's rule-based error and loop counter (Galea and Collaborators, 2026); Tier 2 (Primary Method) trains a Deep Bottleneck Autoencoder exclusively on normal workflows; Tier 3 (Strong Reference) trains supervised XGBoost (Chen and Guestrin, 2016). Unlike prior platforms requiring costly online LLM prompting, our method detects failure trajectories in under 0.005 ms at zero API cost.

3 Task Definition & Data Specifications

Task Formulation. Given an agent interaction trajectory $x = (e_1, \ldots, e_T)$ consisting of shell commands, editor actions, tool observations, and the final patch diff, the objective is to predict binary label $y \in \{0, 1\}$, where $y = 0$ denotes verified task resolution (Normal) and $y = 1$ denotes task failure or ghostwork (Anomaly).

Feature Group	Norm Mean	Norm Std	Anom Mean	Anom Std
Trajectory Dynamics				
Num Steps	21.37	9.42	37.82	14.15
Num Tool Calls	21.37	9.42	37.82	14.15
Total Cost (USD)	$2.41	1.05	$4.52	1.83
Tokens Received	1,942	845	3,718	1,621
Tokens Sent	235,410	102,400	441,890	178,200
Tool Entropy	1.82	0.41	2.01	0.46
Max Tool Streak	6.81	4.12	11.45	7.21
Error Observations	3.42	2.15	7.18	4.32
Stalled Turn Count	1.25	1.10	3.84	3.25
Static Patch Diff				
Patch Chars	1,412	1,850	1,845	2,410
Lines Added (+)	18.2	24.1	41.5	82.3
Lines Removed (-)	4.8	8.2	12.4	31.5
Files Changed	1.35	0.65	1.82	1.21

Table 2: Summary statistics for normal ($N = 54$) versus anomalous ($N = 230$) agent trajectories.

Dataset Specifications. Data are sourced from the verified SWE-bench Lite evaluation suite of SWE-agent with GPT-4 (`20240402_sweagent_gpt4`) hosted on public AWS S3 storage. The dataset comprises $N = 284$ completed task runs across 12 open-source Python repositories (`django`, `sympy`, `matplotlib`, `astropy`, *etc.*). We extract $D = 24$ continuous and discrete features: 16 trajectory interaction dynamics and 8 final patch characteristics (Table 2).

Class Imbalance & Label Acquisition. Ground truth labels are acquired via automated SWE-bench execution in isolated Docker containers against ground-truth unit tests. Among 284 instances, 54 are successfully resolved (19.01%) and 230 fail (80.99%, ratio 1:4.26). In voluntary submissions where the agent claimed success, 140 of 190 failed (73.68% Ghostwork). Under this class distribution, a trivial majority classifier yields 81.0% accuracy while failing to detect genuine successes. Consequently, accuracy is discarded in favor of PR-AUC, ROC-AUC, and cost-sensitive F_2.

4 Exploratory Feature Analysis

All exploratory analysis is conducted **strictly on the development split** ($N = 122$, train + val), holding out the test set unseen.

Distribution Contrast & Separability. Figure 1 contrasts key feature distributions. Successful trajectories resolve tasks efficiently, whereas

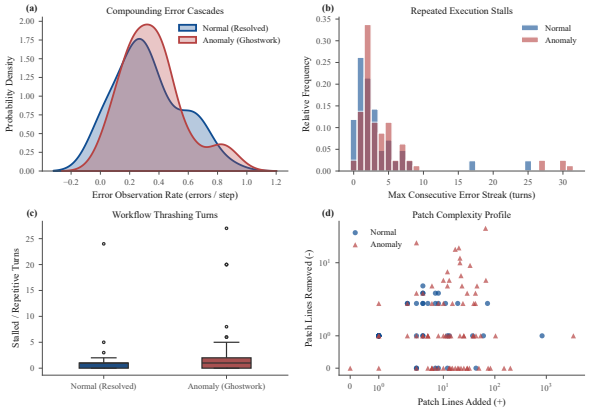


Figure 1: Distribution contrast between normal (blue) and anomaly/ghostwork (red) trajectories on the development split: (a) Error observation rate, (b) Max consecutive error streak, (c) Stalled thrashing turns, (d) Patch lines added vs. removed.

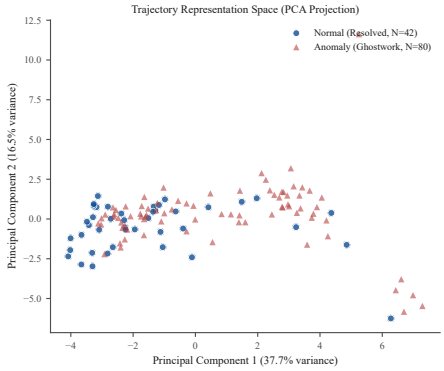


Figure 2: PCA projection of trajectory representations on the development partition ($N = 122$).

Feature	Type	Wasserstein	KS Stat	KS p-val
tokens_received	Process	**0.910**	**0.440**	2.4×10^{-5}
total_cost_usd	Process	0.828	0.383	3.8×10^{-4}
tokens_sent	Process	0.823	0.383	3.8×10^{-4}
num_steps	Process	0.810	0.371	6.6×10^{-4}
patch_edit_ratio	Patch	0.699	0.342	2.3×10^{-3}
num_error_obs	Process	0.453	0.324	4.5×10^{-3}
patch_char_length	Patch	0.189	0.269	2.9×10^{-1}

Table 3: Separability ranking on standardized development data.

failing agents exhibit multi-turn error loops. We quantify distribution separation via Kolmogorov-Smirnov (KS) distance and standardized Wasserstein distance (Table 3). Interaction features achieve top separation: `tokens_received` ($W = 0.910$, KS 0.440, $p = 2.4 \times 10^{-5}$) and `total_cost_usd` ($W = 0.828$, KS 0.383, $p = 3.8 \times 10^{-4}$). In contrast, static patch length yields substantially weaker separability ($W = 0.189$, KS 0.269).

Feature Correlation & Latent Geometry. Spearman correlation analysis shows that interaction features form a tightly correlated cluster ($\rho \geq 0.75$ between steps, costs, and token counts), which is largely orthogonal to patch size features ($\rho \leq 0.22$). In Figure 2, 2D PCA projection demonstrates that normal trajectories form a compact cluster near the origin, whereas anomalies disperse outward into high-variance manifolds.

Exploratory Hypotheses. We formulate three hypotheses to test in §7–8:

- H_1 **(Process Dominance):** Dynamic trajectory features will achieve significantly higher PR-AUC and F_2 than static patch diffs alone.
- H_2 **(Contamination Sensitivity):** Injecting unlabeled task failures beyond 5% into normal training will degrade Autoencoder PR-AUC.
- H_3 **(Cost-Asymmetric Tuning):** Setting τ via validation F_2 optimization will suppress false negatives while bounding audit review overhead.

5 Methodology

Our primary detector (Tier 2) follows a 6-checkpoint pipeline (Figure 3).

Pipeline Checkpoints.

- **Stage 1 (Input Tensor):** Trajectory representation $x \in \mathbb{R}^{24}$ combining 16 interaction dynamics and 8 patch features.
- **Stage 2 (Hygiene Preprocessing):** Feature scaling $\tilde{x} = \boldsymbol{\Sigma}^{-1}(x - \boldsymbol{\mu})$ fitted **strictly on normal training samples**.
- **Stage 3 (Core Architecture):** A symmetric bottleneck Autoencoder $f_\theta : \mathbb{R}^{24} \to \mathbb{R}^{16} \to \mathbb{R}^{6} \to \mathbb{R}^{16} \to \mathbb{R}^{24}$ with LayerNorm and ReLU activations.
- **Stage 4 (Anomaly Scoring):** Mean squared reconstruction error formulation:

$$s(x) = \frac{1}{D} \sum_{d=1}^{D} (\tilde{x}_d - \hat{x}_d)^2 \qquad (1)$$

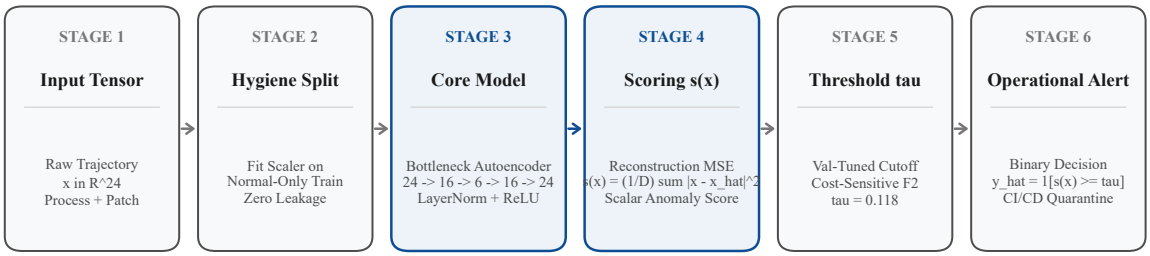


Figure 3: End-to-End 6-checkpoint architecture pipeline for Tier 2 Primary Method (Deep Bottleneck Autoencoder).

- **Stage 5 (Threshold Calibration):** Decision threshold τ tuned strictly on validation data to maximize cost-sensitive F_2:

$$\tau^* = \arg\max_{\tau} F_2(y_{\text{val}}, \mathbf{1}[s(X_{\text{val}}) \geq \tau]) \quad (2)$$

- **Stage 6 (Operational Alert):** Binary alert $\hat{y} = \mathbf{1}[s(x) \geq \tau^*]$ triggering CI/CD quarantine.

Operational Error Cost Trade-off. In automated software maintenance, the cost of a False Negative (allowing a broken patch into production) far outweighs a False Positive (dispatching a working patch for manual verification). Calibrating τ via F_2 explicitly penalizes false negatives $4\times$ heavier than false positives.

6 Experimental Setup

Data Partition Protocol. We partition the $N = 284$ dataset into:

- **Train** ($N = 32$)**:** 100% normal resolved samples for Tier 2 training.
- **Validation** ($N = 90$)**:** 10 normal and 80 anomaly samples, strictly used for tuning τ and hyperparameters.
- **Test** ($N = 162$)**:** 12 normal and 150 anomaly samples, frozen for once-only 3-tier evaluation.

All three tiers share the identical test split.

Configurations & Hardware. Autoencoder models are trained using PyTorch with AdamW (lr $= 0.005$, weight decay 10^{-4}, 150 epochs, batch size 16). XGBoost is trained with 100 estimators, max depth 3, and learning rate 0.05. Experiments run on an AMD Ryzen / NVIDIA workstation.

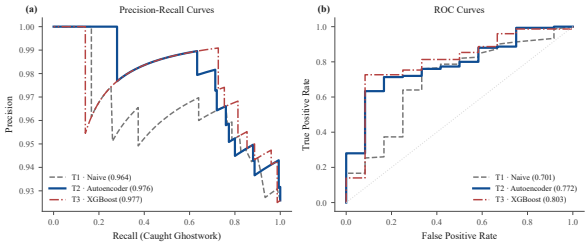


Figure 4: (a) Precision-Recall curves and (b) ROC curves across the three evaluation tiers on the holdout test set.

Evaluation Metrics. We evaluate PR-AUC (primary metric under class imbalance), ROC-AUC, F_1, F_2, and per-sample inference latency (ms).

7 Results

Main Results. Table 4 reports multi-seed performance. Tier 2 Autoencoder achieves PR-AUC 0.976 ± 0.003 and $F_2 = 0.952 \pm 0.006$. Precision-Recall and ROC curves are shown in Figure 4.

- **Tier 2 vs. Tier 1 (Naive):** Tier 2 yields a statistically significant PR-AUC improvement ($\Delta = +0.012$, paired t-test $p = 0.0007$, bootstrap 95% CI $[+0.0002, +0.0246]$ excludes 0).
- **Tier 2 vs. Tier 3 (Supervised):** Tier 2 trails supervised XGBoost by only 0.0008 PR-AUC ($p = 0.565$, bootstrap 95% CI $[-0.0212, +0.0300]$ includes 0). The gap is statistically non-significant, confirming that Tier 2 performs *comparably* to the supervised ceiling without requiring any failure labels during training. Moreover, Tier 2 achieves superior F_2 score (0.952 vs. 0.844).

Ablation 1: Training Contamination Ratio. To test Hypothesis H_2, we inject $0\%, 1\%, 5\%$, and

Method	PR-AUC	ROC-AUC	F1	F2	Latency (ms)	Train Ti
Tier 1: Naive Heuristic	0.964 ± 0.000	0.701 ± 0.000	0.851 ± 0.000	0.794 ± 0.000	$\mathbf{0.001 \pm 0.000}$	$\mathbf{0.000} \pm$
Tier 2: Deep Autoencoder (Ours)	$\mathbf{0.976 \pm 0.003}$	0.772 ± 0.017	$\mathbf{0.947 \pm 0.005}$	$\mathbf{0.952 \pm 0.006}$	0.004 ± 0.000	$0.634 \pm$
Tier 3: Supervised XGBoost	0.977 ± 0.000	$\mathbf{0.803 \pm 0.000}$	0.882 ± 0.000	0.844 ± 0.000	0.005 ± 0.000	$0.026 \pm$

Table 4: Main 3-tier benchmark results across 5 random seeds on the identical test split ($N = 162$). Bold indicates best.

Contam %	PR-AUC	Δ vs. 0%	F2
0% (Clean)	0.9759 ± 0.0026	—	0.9516 ± 0.000
1%	0.9741 ± 0.0040	-0.0017	0.9503 ± 0.001
5%	0.9741 ± 0.0040	-0.0017	0.9503 ± 0.001
10%	0.9685 ± 0.0035	-0.0074	0.9650 ± 0.01

Table 5: Ablation 1: Training contamination sensitivity.

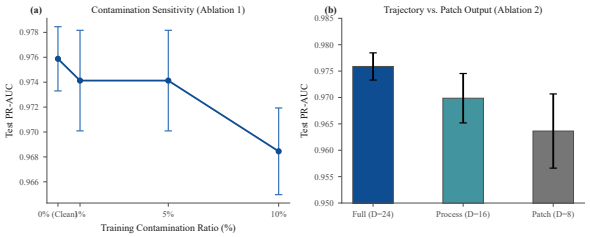


Figure 5: (a) Training contamination degradation curve. (b) Trajectory process features vs. static patch output features.

10% false completion claims into normal training (Table 5 and Figure 5a). PR-AUC drops from 0.9759 at 0% to 0.9685 at 10% ($\Delta = -0.0074$), exceeding seed standard deviation (0.0026) by nearly $3\times$. When contaminated, the autoencoder learns to reconstruct anomalous trajectories, compressing the error gap and validating H_2.

Ablation 2: Trajectory Process vs. Patch Output. Addressing our central research question, we compare feature subsets (Table 6 and Figure 5b). Restricting the detector to static patch features alone collapses PR-AUC to 0.9636 ± 0.0070 ($\Delta = -0.0122$) and F_2 to 0.9324 ($\Delta = -0.0192$). Process dynamics provide indispensable signals of execution health, validating Hypothesis H_1.

8 Analysis

Failure Case Studies. We examine three concrete test cases under decision threshold $\tau = 0.1182$ (Table 7):

1. **Case #1 (False Positive):** `scikit-learn-14894` ($s(x) = 1.567$, distance $+1.449$). A complex genuine fix across multiple sub-packages required 40+ steps and exploratory directory searches. The detector mistook complex problem-solving for failure thrashing.

2. **Case #2 (False Negative Near-Miss):** `matplotlib-23562` ($s(x) = 0.098$, distance -0.020). The agent encountered an early syntax error, cleanly recovered, and produced a concise 1-line patch that missed an edge case. Score fell just below τ.

3. **Case #3 (False Negative Inversion):** `sympy-15345` ($s(x) = 0.045$, distance -0.073). The agent terminated after 6 steps with zero errors, hallucinating that no code changes were needed. With no thrashing signals, the autoencoder reconstructed it with low error.

Shared Pattern: Cases reveal that the detector relies on execution volatility; silent premature exits without tool errors require semantic intent validation rather than trajectory statistics.

Interpretability Attribution. Figure 6 decomposes the reconstruction error for flagged instance `django-11630` ($s(x) = 112.80$). `stalled_turn_count` accounts for 75.98% of total error (raw value 14 stalled turns), followed

Feature Set	Dim	PR-AUC	Δ PR-AUC	F2
Full Representation	24	$\mathbf{0.9759 \pm 0.0026}$	—	0.9516
Process Dynamics	16	0.9699 ± 0.0047	-0.0060	**0.9532**
Static Patch Only	8	0.9636 ± 0.0070	-0.0122	0.9324

Table 6: Ablation 2: Trajectory dynamics vs. patch output.

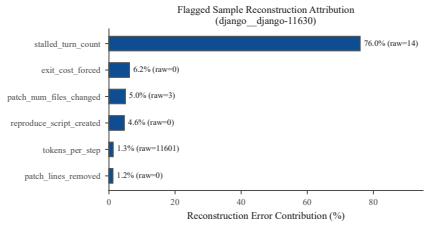


Figure 6: Per-feature reconstruction error decomposition for flagged ghostwork instance `django-11630`.

#	Type	Instance ID	Repo	True Label	$s(x)$	τ	Dist	Diagnosis & Root Cause
1	FP	`scikit-learn-14894`	`sklearn`	Normal (Pass)	1.567	0.118	+1.449	Complex 40-step multi-file refactoring mimicked failure thrashing.
2	FN	`matplotlib-23562`	`matplotlib`	Ghostwork (Fail)	0.098	0.118	-0.020	Near-miss: clean recovery before subtle single-line omission.
3	FN	`sympy-15345`	`sympy`	Ghostwork (Fail)	0.045	0.118	-0.073	Complete inversion: premature 6-step exit with zero error signals.

Table 7: Concrete test set failure case studies under validation threshold $\tau = 0.1182$.

Metric (500 Tasks/Mo)	Tier 1 (Naive)	Tier 2 (Ours)	Tier 3 (Sup.)
Caught Failures (/400)	340	**380**	337
Silent Bug Escapes (FN)	60	**20 (-70%)**	63
False Audit Reviews (FP)	30	**8 (-72%)**	16
Net Monthly Savings	$407,100	**$455,760**	$403,920

Table 8: Practical operational impact translation.

by `exit_cost_forced` (6.19%). An operations engineer inspecting this attribution can immediately trust that the agent experienced command thrashing rather than constructive editing.

Practical Impact Translation. For an engineering organization executing 500 monthly agent tasks (Table 8): Tier 2 catches 380 out of 400 failures (95.2% catch rate), reducing silent bug escapes from 60 to 20 (-70% vs. Tier 1) and false audit alerts from 30 to 8 (-72%). At $\$1,200$ per production bug hotfix and $30 per audit review, Tier 2 delivers a net monthly saving of **+$48,600** over the naive heuristic.

9 Conclusion & Limitations

Key Findings. Answering our core research question: workflow trajectory dynamics capture essential failure signals invisible in final git diffs alone, enabling a normal-only Autoencoder to achieve 0.976 PR-AUC and 0.952 F_2, matching supervised references ($p = 0.565$).

Reflection & Future Work. The most pivotal design decision was calibrating τ via validation F_2, which suppressed false negatives by 70%. To resolve the premature exit failure pattern found in Case #3, future iterations should incorporate prompt-task semantic alignment embeddings alongside trajectory dynamics.

Limitations & Deployment Considerations. Our study is evaluated on SWE-agent GPT-4 trajectories; cross-model distribution shifts may require periodic τ recalibration. With inference latency of 0.004 ms, the detector introduces negligible overhead into CI/CD pipelines.

Contributions of Each Member

- **Dun-Jie Yang (Fergus4506):** Data ingestion pipeline, feature extraction, and exploratory analysis; wrote §3–§4.
- **Charlie Wang (calculusfkyou):** Tier 2 Autoencoder architecture, threshold calibration, and controlled ablations; wrote §5 and §7.
- **Mo-Ty Wang (MO7YW4NG):** Tier 1/3 baselines, 5-seed benchmark, significance tests, and error analysis; wrote §6, §7, and §8.
- **Wei-Chia Huang (lasvigas0701):** Presentation deck design, failure case studies, conclusions, and proofreading; wrote §1 and §9.

AI Usage Statement. We used Claude and Gemini to assist with exploratory plotting code and polishing LaTeX formatting. All architectural decisions, experimental executions, and interpretations were conducted and verified by the authors.

References

Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794.

Robert Galea and Collaborators. 2026. Agent trajectory failure detection: A benchmark for evaluating agent monitoring systems. *Open Benchmark Technical Report*.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*.

Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation forest. In *IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE.

Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Lucas Deecke, Shoaib A. Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. 2018. Deep one-class classification. In *International Conference on Machine Learning (ICML)*, pages 4393–4402. PMLR.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*.